Operadores y Estructuras de Decisión en Java con IntelliJ IDEA

Taller Práctico Intensivo

Versión resuelta · Proyecto Taller_Java · Java 17 · IntelliJ IDEA Community Edition

De:Joysmer Leonel Escalona Peinado

Grupo:M4

1. Guía de Inicio y Configuración del Entorno en IntelliJ IDEA

El dominio de la lógica de control constituye el verdadero motor del desarrollo de software en Java. Un profesional de la programación no solo escribe código ejecutable, sino que diseña estructuras capaces de tomar decisiones dinámicas, evaluar condiciones bajo algoritmos eficientes y garantizar el flujo correcto de los datos en tiempo de ejecución. Para sentar las bases sólidas de este taller práctico, configuraremos desde cero un entorno de desarrollo profesional en IntelliJ IDEA.

Instrucciones Paso a Paso para la Creación del Proyecto:

1. **Apertura del Entorno:** Inicie IntelliJ IDEA y seleccione la opción New Project (Nuevo Proyecto) en la pantalla de bienvenida o a través de File > New > Project.
2. **Configuración de Parámetros Generales:**
   - **Name (Nombre):** Ingrese exactamente `TallerUnidad1_2`.
   - **Location (Ubicación):** Elija un directorio de trabajo local con permisos de escritura.
   - **Language (Lenguaje):** Seleccione Java.
   - **Build System (Sistema de Construcción):** Seleccione IntelliJ.
   - **JDK (Java Development Kit):** Verifique que esté seleccionada una versión LTS activa de Java (JDK 17 o superior recomendada).
3. **Estructuración del Paquete Principal:**
   - En la ventana del proyecto (Project Tool Window), expanda la raíz y localice la carpeta `src`.
   - Haga clic derecho sobre `src`, seleccione New > Package y escriba el nombre estructurado del paquete: `com.taller.practico`.
4. **Creación de la Clase de Entrada:**

- Haga clic derecho sobre el paquete recien creado (`com.taller.practico`), seleccione New > Java Class y defina la clase con el nombre `Main`.

Tips de Productividad en IntelliJ IDEA

> **Atajo `psvm` / `main`:** Dentro del cuerpo de la clase `Main`, escriba `psvm` (o `main`) y presione Tab o Enter. El IDE generará instantáneamente el bloque completo de la firma del método principal: `public static void main(String[] args)`.
>
> **Atajo `sout`:** Escriba `sout` y presione Tab dentro de cualquier método para generar automáticamente la sentencia de impresión estándar `System.out.println();`.
>
> **Compilación y Ejecución Rápida:** Presione la combinación de teclas Shift + F10 en Windows/Linux (o Control + R en macOS) para compilar y ejecutar el archivo actual sin recurrir al ratón.

Una vez configurado nuestro entorno de desarrollo e inicializada la clase principal, estamos listos para pasar de la infraestructura básica a la evaluación precisa de expresiones matemáticas y lógicas.

✓ SOLUCIÓN: Sección 1 (Entorno)

Parámetro	Valor usado en el proyecto
Nombre del proyecto	Taller_Java (el enunciado pide `TallerUnidad1_2`; se renombra con clic derecho > Refactor > Rename si el docente lo exige)
Ubicación	~/Descargas/SpringBoot/LogiTrack_SprinBoot/ Taller_Java
Lenguaje / Build System	Java / IntelliJ
JDK	Java 17 (LTS)
Paquete	com.taller.practico (src > New > Package)
Clase de entrada	Main (con `psvm` + Tab para el método principal y `sout` + Tab para imprimir)

Estructura final del proyecto

```
Taller_Java/
└── src/
    └── com/taller/practico/
        ├── Main.java
        ├── ClasificacionEdades.java
        ├── SistemaAprobacion.java
        ├── TarifasEntrada.java
        ├── OperadorTernario.java
        ├── DiaSemanaTradicional.java
        ├── DiaSemanaModerno.java
        ├── SimuladorCajaInventario.java
        ├── ErrorAsignacion.java / ErrorAsignacionCorregido.java
```

```
        ├── ErrorComparacionString.java / ErrorComparacionStringCorregido.java
        ├── ErrorFallThrough.java / ErrorFallThroughCorregido.java
        └── ErrorPrecision.java / ErrorPrecisionCorregido.java
```

2. Sección 1: Calentamiento de Operadores

En esta primera fase analítica, evaluaremos la precedencia de operadores, el comportamiento del operador residuo (`%`), las diferencias fundamentales entre el incremento prefijo (`++x`) y posfijo (`x++`), la promoción implícita de tipos de datos en operaciones aritméticas y la evaluación con cortocircuito (short-circuit) en operadores lógicos (`&&`, `||`, `!`).

Nota Pedagógica de Estado: Para la evaluación de la siguiente tabla, asuma de manera independiente para cada fila que las variables parten del estado inicial: `int x = 7;` y `boolean flag = false;`. Complete la columna de predicción mediante análisis mental o en papel antes de trasladar el código a su clase `Main` en IntelliJ IDEA.

Tabla de Evaluación de Expresiones

Expresión en Java	Predicción del Estudiante (Sin IDE)	Resultado Real en IntelliJ IDEA
`int seg = 86400 % 3600;`	0	seg = 0
`int y = ++x + x++;`	16 (x queda en 9)	y = 16, x = 9
`double div1 = 15 / 2;` `double div2 = 15.0 / 2;`	div1 = 7.0 y div2 = 7.5	div1 = 7.0, div2 = 7.5
`boolean res = flag && (++x > 0);`	false (x sigue en 7)	res = false, x = 7

Está estrictamente prohibido ejecutar la tabla antes de registrar por escrito la predicción personal.

Comprender con exactitud cómo el compilador y la Máquina Virtual de Java (JVM) evalúan los operadores y administran el estado de las variables en memoria es el pilar fundamental sobre el cual se construyen los condicionales y la toma de decisiones complejas.

✔ SOLUCIÓN: Sección 1 (Operadores)

Razonamiento de cada predicción

- **`86400 % 3600`:** 86400 es múltiplo exacto de 3600 (3600 × 24), así que el residuo es 0.

- **`++x + x++`:** con x = 7, el prefijo `++x` incrementa primero y vale 8. Luego `x++` devuelve el valor actual (8) y después incrementa x a 9. La suma es 8 + 8 = 16 y x termina en 9.
- **`15 / 2` y `15.0 / 2`:** con dos enteros la división es entera y da 7, que al asignarse a `double` se convierte en 7.0. Con `15.0` uno de los operandos es `double` (promoción implícita), así que la división es decimal: 7.5.
- **`flag && (++x > 0)`:** `flag` es `false`, y en un `&&` el resultado ya es `false`. Java hace cortocircuito y no evalúa `++x > 0`, por eso x sigue valiendo 7.

Main.java

```
package com.taller.practico;

public class Main {
    public static void main(String[] args) {
        int seg = 86400 % 3600;
        System.out.println("seg = " + seg);

        int x = 7;
        int y = ++x + x++;
        System.out.println("y = " + y + ", x = " + x);

        double div1 = 15 / 2;
        double div2 = 15.0 / 2;
        System.out.println("div1 = " + div1);
        System.out.println("div2 = " + div2);

        x = 7;
        boolean flag = false;
        boolean res = flag && (++x > 0);
        System.out.println("res = " + res + ", x = " + x);
    }
}
```

Salida en consola

```
seg = 0
y = 16, x = 9
div1 = 7.0
div2 = 7.5
res = false, x = 7
```

3. Sección 2: Decisiones con if-else y Operador Ternario

Las estructuras condicionales permiten bifucar el flujo de un programa en función de la veracidad de una o más condiciones booleanas. A continuación, resolverá tres retos contextualizados en escenarios reales del desarrollo de software.

Reto 1: Clasificación de Edades para Acceso a Plataforma Educativa

Diseñe una lógica condicional en cascada (`if-else if-else`) que categorice a los usuarios de una plataforma de aprendizaje en línea según su edad, determinando los permisos de perfil y la retención de datos asociada.

- **Requisitos de Entrada:** Una variable de tipo entero denominada `edadUsuario` (ej. 10, 15, 28, 68).
- **Condiciones Lógicas:**
  - Si `edadUsuario` es menor a 12, clasificar como "Infante" (requiere tutor de cuenta).
  - Si `edadUsuario` está entre 12 y 17 (inclusive), clasificar como "Adolescente" (acceso restringido a foros comunitarios).
  - Si `edadUsuario` está entre 18 y 64 (inclusive), clasificar como "Adulto" (acceso a funcionalidades estándar).
  - Si `edadUsuario` es mayor o igual a 65, clasificar como "Adulto Mayor" (acceso con interfaz adaptada).
- **Salida Esperada:** Impresión en consola indicando la categoría asignada y el perfil de acceso activado.

✔ SOLUCIÓN: Reto 1

ClasificacionEdades.java

```
package com.taller.practico;

public class ClasificacionEdades {
    public static void main(String[] args) {
        int edadUsuario = 28;

        if (edadUsuario < 12) {
            System.out.println("Categoría: Infante — requiere tutor de cuenta.");
        } else if (edadUsuario <= 17) {
            System.out.println("Categoría: Adolescente — acceso restringido a foros
comunitarios.");
        } else if (edadUsuario <= 64) {
            System.out.println("Categoría: Adulto — acceso a funcionalidades
estándar.");
        } else {
```

```
            System.out.println("Categoría: Adulto Mayor — acceso con interfaz
adaptada.");
        }
    }
}
```

Salida en consola (edadUsuario = 28)

```
Categoría: Adulto — acceso a funcionalidades estándar.
```

Resultado con las edades de ejemplo del enunciado:

edadUsuario	Salida
10	Categoría: Infante — requiere tutor de cuenta.
15	Categoría: Adolescente — acceso restringido a foros comunitarios.
28	Categoría: Adulto — acceso a funcionalidades estándar.
68	Categoría: Adulto Mayor — acceso con interfaz adaptada.

Como el `else if` solo se evalúa cuando el anterior falló, no hace falta repetir el límite inferior de cada rango: si llega a `edadUsuario <= 17` ya se sabe que es 12 o más.

Reto 2: Sistema de Aprobación de Notas y Asistencia

Desarrolle un evaluador de rendimiento académico para validar si un estudiante aprueba un curso de programación considerando tanto el puntaje obtenido como su porcentaje de asistencia obligatoria.

- **Requisitos de Entrada:** Una variable de tipo decimal `calificacionFinal` (de 0.0 a 100.0), una variable de tipo entero `porcentajeAsistencia` (de 0 a 100), y una variable booleana `entregoProyectoFinal`.
- **Condiciones Lógicas:**
  - Un estudiante aprueba el curso si su `calificacionFinal` es mayor o igual a 70.0 Y (`&&`) su `porcentajeAsistencia` es mayor o igual al 80%.
  - **Excepción de recuperación:** Si no cumple la asistencia o la nota mínima, pero `entregoProyectoFinal` es verdadero (`true`) Y la `calificacionFinal` es al menos 60.0, el estado será "Pendiente de Examen Extraordinario".
  - Si no cumple ninguna de las anteriores, el estado será "Reprobado".
- **Salida Esperada:** Un mensaje claro por consola especificando el dictamen académico del alumno.

✔ SOLUCIÓN: Reto 2

SistemaAprobacion.java

```
package com.taller.practico;

public class SistemaAprobacion {
    public static void main(String[] args) {
        double calificacionFinal = 75.0;
        int porcentajeAsistencia = 85;
        boolean entregoProyectoFinal = false;

        if (calificacionFinal >= 70.0 && porcentajeAsistencia >= 80) {
            System.out.println("Estado: APROBADO");
        } else if (entregoProyectoFinal && calificacionFinal >= 60.0) {
            System.out.println("Estado: Pendiente de Examen Extraordinario");
        } else {
            System.out.println("Estado: REPROBADO");
        }
    }
}
```

Salida en consola (75.0, 85, false)

```
Estado: APROBADO
```

Prueba de los tres caminos posibles:

calificacionFinal	porcentajeAsistencia	entregoProyectoFinal	Dictamen
75.0	85	false	APROBADO
65.0	70	true	Pendiente de Examen Extraordinario
50.0	90	false	REPROBADO

Reto 3: Cálculo de Tarifas de Entrada y Descuentos

Cree un programa que determine el precio de venta de una entrada para un evento tecnológico en función del tipo de boleto adquirido.

- **Requisitos de Entrada:** Un valor de tipo `String` denominado `tipoEntrada` ("VIP", "ESTUDIANTE", "GENERAL", "ADULTO_MAYOR") y un valor base de entrada `precioBase = 100.0`.
- **Condiciones Lógicas:**
  - Boleto "VIP": Recibe un 10% de recargo sobre el precio base por beneficios exclusivos.

  - Boleto "ESTUDIANTE": Recibe un 50% de descuento sobre el precio base.
  - Boleto "ADULTO_MAYOR": Recibe un 40% de descuento sobre el precio base.
  - Boleto "GENERAL": Se cobra la tarifa base sin modificaciones.
- **Salida Esperada:** Imprimir en consola el desglose del tipo de entrada y el monto final a pagar.

✔ SOLUCIÓN: Reto 3

TarifasEntrada.java

```
package com.taller.practico;

import java.util.Locale;

public class TarifasEntrada {
    public static void main(String[] args) {
        String tipoEntrada = "VIP";
        double precioBase = 100.0;
        double precioFinal;

        if (tipoEntrada.equals("VIP")) {
            precioFinal = precioBase * 1.10;
        } else if (tipoEntrada.equals("ESTUDIANTE")) {
            precioFinal = precioBase * 0.50;
        } else if (tipoEntrada.equals("ADULTO_MAYOR")) {
            precioFinal = precioBase * 0.60;
        } else {
            precioFinal = precioBase;
        }

        System.out.println("Tipo de entrada: " + tipoEntrada);
        System.out.println("Monto final a pagar: $" + String.format(Locale.US,
"%.2f", precioFinal));
    }
}
```

Salida en consola (tipoEntrada = "VIP")

```
Tipo de entrada: VIP
Monto final a pagar: $110.00
```

tipoEntrada	Operación	Monto final
VIP	100.0 × 1.10	$110.00
ESTUDIANTE	100.0 × 0.50	$50.00
ADULTO_MAYOR	100.0 × 0.60	$60.00
GENERAL	100.0	$100.00

Las cadenas se comparan con `equals`, no con == (se explica en el Fragmento 2 de la Sección 5).

Subsección de Simplificación: Refactorización a Operador Ternario

El operador ternario (`condición ? expresión_si_verdadero : expresión_si_falso`) permite evaluar condicionales binarios simples de manera concisa en una sola línea de código.

Examine el siguiente código tradicional:

```
String estadoConexion;
if (cookieSesionActiva == true) {
    estadoConexion = "ACTIVA";
} else {
    estadoConexion = "INACTIVA";
}
```

> **Consigna de Refactorización:** Escriba en su clase de práctica una única línea de código equivalente a la estructura anterior, sustituyendo todo el bloque `if-else` por una asignación directa utilizando el operador ternario.

Dominar las evaluaciones binarias mediante `if-else` y operadores ternarios proporciona una flexibilidad enorme; sin embargo, cuando nos enfrentamos a múltiples alternativas discretas sobre un mismo elemento, Java ofrece la estructura de selección múltiple `switch`.

✔ SOLUCIÓN: Operador ternario

La línea pedida es:

```
String estadoConexion = cookieSesionActiva ? "ACTIVA" : "INACTIVA";
```

OperadorTernario.java

```
package com.taller.practico;

public class OperadorTernario {
    public static void main(String[] args) {
        boolean cookieSesionActiva = true;
        String estadoConexion = cookieSesionActiva ? "ACTIVA" : "INACTIVA";
        System.out.println("Estado de conexión: " + estadoConexion);
    }
}
```

Salida en consola

```
Estado de conexión: ACTIVA
```

Como `cookieSesionActiva` ya es booleana, tampoco hace falta compararla con `== true`: la variable misma es la condición.

4. Sección 3: Selección Múltiple con switch (Tradicional y Moderno)

Caso Práctico: Gestor de Días de la Semana y Tipo de Jornada

En las aplicaciones empresariales es habitual transformar valores numéricos o códigos discretos en días hábiles o tipos de jornada operativa. Se solicita implementar un programa interactivo que procese el número de un día de la semana (1 para Lunes, 7 para Domingo) y determine el tipo de jornada laboral correspondiente.

Requisitos de Implementación:

El estudiante debe escribir en la clase `Main` dos versiones independientes que resuelvan la misma lógica de negocio:

1. **Implementación 1 (Sintaxis Tradicional):** Uso de la instrucción `switch` clásica con etiquetas `case`, sentencias de interrupción explícitas `break` y un caso `default` para manejar entradas inválidas.
2. **Implementación 2 (Sintaxis Moderna):** Uso de la expresión `switch` introducida en versiones recientes de Java, empleando la notación de flecha (`->`).

Explicación Pedagógica de la Sintaxis Moderna: La notación de flecha (`->`) elimina estructuralmente el riesgo de caída en cascada (fall-through) involuntaria al no requerir la palabra clave `break`. Además, permite retornar un valor directamente a una variable, convirtiendo el `switch` de una sentencia de control (statement) a una expresión evaluable (expression).

Tabla Comparativa de Sintaxis switch

Criterio de Comparación	Sintaxis Tradicional (switch)	Sintaxis Moderna (switch con ->)
Mecanismo de Control de Flujo	Requiere la instrucción `break` al final de cada caso; de lo contrario, el flujo cae en cascada ejecutando los casos subsiguientes.	No requiere instrucción `break`. Solamente se ejecuta la expresión o bloque directamente asociado a la derecha de la flecha.
Concisión y Retorno de Valores	Funciona principalmente como bloque de sentencias. Asignar un resultado exige declarar una variable previa y mutarla en cada `case`.	Puede actuar como expresión que retorna un valor directamente, permitiendo inicializar variables de forma inmutable (`final`) y limpia.

Integrar operadores aritméticos, evaluaciones condicionales compuestas con `if-else` y selecciones múltiples con `switch` es el paso natural para la construcción de sistemas empresariales robustos.

✔ SOLUCIÓN: Sección 3 (switch)

> El enunciado menciona la clase `Main`, pero para tener cada reto en su propio `main` las dos versiones están en clases independientes: `DiaSemanaTradicional` y `DiaSemanaModerno`.

Implementación 1: DiaSemanaTradicional.java

```
package com.taller.practico;

public class DiaSemanaTradicional {
    public static void main(String[] args) {
        int numeroDia = 6;
        String tipoJornada;

        switch (numeroDia) {
            case 1:
            case 2:
            case 3:
            case 4:
            case 5:
                tipoJornada = "Jornada laboral";
                break;
            case 6:
                tipoJornada = "Jornada reducida";
                break;
            case 7:
                tipoJornada = "Descanso";
                break;
            default:
                tipoJornada = "Día inválido";
        }

        System.out.println("Día " + numeroDia + " (tradicional): " + tipoJornada);
    }
}
```

Salida en consola

```
Día 6 (tradicional): Jornada reducida
```

Implementación 2: DiaSemanaModerno.java

```
package com.taller.practico;

public class DiaSemanaModerno {
```

```
    public static void main(String[] args) {
        int numeroDia = 6;

        String tipoJornada = switch (numeroDia) {
            case 1, 2, 3, 4, 5 -> "Jornada laboral";
            case 6             -> "Jornada reducida";
            case 7             -> "Descanso";
            default            -> "Día inválido";
        };

        System.out.println("Día " + numeroDia + " (moderno): " + tipoJornada);
    }
}
```

Salida en consola

```
Día 6 (moderno): Jornada reducida
```

numeroDia	tipoJornada
1, 2, 3, 4, 5	Jornada laboral
6	Jornada reducida
7	Descanso
cualquier otro	Día inválido (caso `default`)

Ambas versiones dan el mismo resultado. En la tradicional, los `case 1` a `case 4` quedan vacíos a propósito para que caigan al `case 5` (agrupación); en la moderna se agrupan con comas: `case 1, 2, 3, 4, 5 ->`.

5. Sección 4: Reto Integrador: Simulador Práctico de Caja e Inventario

Escenario Empresarial: Sistema de Facturación y Liquidación en Tienda Tecnológica

Usted ha sido designado para construir el motor principal de liquidación e inventario para un establecimiento comercial de tecnología y licencias de software. El sistema debe calcular el costo total de una compra, aplicar descuentos condicionales por tipo de cliente, procesar cupones de descuento adicionales y calcular los impuestos de ley (IVA).

Reglas de Negocio del Sistema:

1. **Cálculo del Subtotal Base:**
   - Cada unidad o paquete de licencias tiene un costo unitario estándar de $250.00.

   - El subtotal base se calcula mediante la multiplicación aritmética: `cantidad * 250.00`.
2. **Descuento por Categoría de Cliente (vía `switch`):**
   - Cliente "VIP": Recibe un 20% de descuento sobre el subtotal base.
   - Cliente "REGULAR": Recibe un 10% de descuento sobre el subtotal base.
   - Cliente "NUEVO": Recibe un 5% de descuento sobre el subtotal base.
   - Cualquier otra categoría no recibe descuento comercial (0%).
3. **Validación de Cupón Promocional y Monto Mínimo (vía `if-else` y operadores lógicos):**
   - Si el cliente presenta el código de cupón "PROMO1 año" Y el subtotal tras el primer descuento es mayor o igual a 500.00, se otorga un descuento fijo adicional de **50.00**.
   - Si no cumple ambas condiciones simultáneamente, el descuento por cupón es $0.00.
4. **Cálculo de Impuestos y Monto Final:**
   - Sobre la base imponible resultante (subtotal menos todos los descuentos aplicados), se calcula el 16% de IVA (`monto * 0.16`).
   - El total final a cobrar es la suma de la base imponible más el IVA calculado.

Casos de Prueba (Test Cases) con Resultados Esperados:

El estudiante debe configurar los siguientes datos de entrada directamente en su código para validar la exactitud técnica de su programa:

Caso de Prueba A:

- **Datos de Entrada:** `cantidad = 3`, `tipoCliente = "VIP"`, `cupon = "PROMO1 año"`.
- **Desglose Pasos de Cálculo:**
  - Subtotal Base: 3 × $250.00 = $750.00
  - Descuento VIP (20%): −$150.00 → Subtotal con descuento = $600.00
  - Validación de Cupón: Es "PROMO1 año" y $600.00 ≥ $500.00 → Aplica −$50.00
  - Base Imponible: $600.00 − $50.00 = $550.00
  - IVA (16% de $550.00): +$88.00
- **Resultado Esperado Final a Imprimir:** Total a pagar = $638.00

Caso de Prueba B:

- **Datos de Entrada:** `cantidad = 1`, `tipoCliente = "NUEVO"`, `cupon = "PROMO1 año"`.
- **Desglose Pasos de Cálculo:**

  - Subtotal Base: 1 × $250.00 = $250.00
  - Descuento NUEVO (5%): −$12.50 → Subtotal con descuento = $237.50
  - Validación de Cupón: Es "PROMO1 año" pero $237.50 < $500.00 → NO aplica ($0.00)
  - Base Imponible: $237.50
  - IVA (16% de $237.50): +$38.00
- **Resultado Esperado Final a Imprimir:** Total a pagar = $275.50

✔ SOLUCIÓN: Reto Integrador

El `main` llama dos veces al método `facturar` con los datos de los casos A y B. El descuento por cliente usa `switch` moderno y el cupón usa `if` con `&&`.

SimuladorCajaInventario.java

```
package com.taller.practico;

import java.util.Locale;

public class SimuladorCajaInventario {
    public static void main(String[] args) {
        facturar(3, "VIP", "PROMO1 año");
        facturar(1, "NUEVO", "PROMO1 año");
    }

    static void facturar(int cantidad, String tipoCliente, String cupon) {
        double subtotalBase = cantidad * 250.00;

        double porcentajeDescuento = switch (tipoCliente) {
            case "VIP"     -> 0.20;
            case "REGULAR" -> 0.10;
            case "NUEVO"   -> 0.05;
            default        -> 0.0;
        };

        double descuentoCliente = subtotalBase * porcentajeDescuento;
        double subtotalConDescuento = subtotalBase - descuentoCliente;

        double descuentoCupon = 0.0;
        if (cupon.equals("PROMO1 año") && subtotalConDescuento >= 500.00) {
            descuentoCupon = 50.00;
        }

        double baseImponible = subtotalConDescuento - descuentoCupon;
        double iva = baseImponible * 0.16;
        double totalFinal = baseImponible + iva;

        System.out.println("── Factura ──");
```

```
        System.out.println("Cantidad: " + cantidad + " | Cliente: " + tipoCliente +
" | Cupón: " + cupon);
        System.out.println("Subtotal base: $" + String.format(Locale.US, "%.2f",
subtotalBase));
        System.out.println("Descuento cliente (" + (porcentajeDescuento * 100) +
"%): -$" + String.format(Locale.US, "%.2f", descuentoCliente));
        System.out.println("Subtotal con descuento: $" + String.format(Locale.US,
"%.2f", subtotalConDescuento));
        System.out.println("Descuento cupón: -$" + String.format(Locale.US, "%.2f",
descuentoCupon));
        System.out.println("Base imponible: $" + String.format(Locale.US, "%.2f",
baseImponible));
        System.out.println("IVA (16%): $" + String.format(Locale.US, "%.2f", iva));
        System.out.println("Total a pagar = $" + String.format(Locale.US, "%.2f",
totalFinal));
        System.out.println();
    }
}
```

Salida en consola

```
── Factura ──
Cantidad: 3 | Cliente: VIP | Cupón: PROMO1 año
Subtotal base: $750.00
Descuento cliente (20.0%): -$150.00
Subtotal con descuento: $600.00
Descuento cupón: -$50.00
Base imponible: $550.00
IVA (16%): $88.00
Total a pagar = $638.00

── Factura ──
Cantidad: 1 | Cliente: NUEVO | Cupón: PROMO1 año
Subtotal base: $250.00
Descuento cliente (5.0%): -$12.50
Subtotal con descuento: $237.50
Descuento cupón: -$0.00
Base imponible: $237.50
IVA (16%): $38.00
Total a pagar = $275.50
```

Caso	Esperado	Obtenido	Resultado
A (3, VIP, PROMO1 año)	$638.00	$638.00	✔ Correcto
B (1, NUEVO, PROMO1 año)	$275.50	$275.50	✔ Correcto

> Se imprime también el desglose completo para poder comparar cada paso con el enunciado (subtotal, descuento, cupón, base imponible e IVA).

6. Sección 5: Análisis y Depuración de Código (Debugging en IntelliJ IDEA)

Incluso cuando la lógica algorítmica parece correcta en la fase de diseño, durante la escritura de código en Java pueden introducirse fallas sintácticas o de tipo. Saber diagnosticar y corregir estos inconvenientes requiere dominar el depurador (Debugger) del entorno de desarrollo.

A continuación, se presentan cuatro fragmentos independientes de código Java que contienen errores clásicos en el desarrollo de software. Copie cada fragmento en clases independientes dentro de su paquete `com.taller.practico`, utilice el depurador de IntelliJ IDEA para inspeccionar las variables y aplique las correcciones requeridas.

Fragmento 1: Falla en Condición Lógica (Asignación vs Comparación)

```
package com.taller.practico;

public class ErrorAsignacion {
    public static void main(String[] args) {
        boolean cookieActiva = false;

        // PUNTO DE INSPECCIÓN: Analice el operador utilizado dentro del parentesis
del 'if'
        if (cookieActiva = true) {
            System.out.println("Estado de sesión: Sesión del usuario activa.");
        } else {
            System.out.println("Estado de sesión: Sesión inactiva.");
        }
    }
}
```

✔ SOLUCIÓN: Fragmento 1

- **Error:** dentro del `if` se usó `=` (asignación) en lugar de `==` (comparación). La expresión `cookieActiva = true` asigna `true` a la variable y devuelve `true`, así que el `if` siempre se cumple.
- **Qué se ve en el depurador:** al avanzar con F8 sobre la línea del `if`, la pestaña Variables muestra `cookieActiva = true` aunque se inicializó en `false`.
- **Corrección:** comparar con `==`. Como la variable ya es booleana, también basta escribir `if (cookieActiva)`.

Salida con el error (debería decir "Sesión inactiva")

```
Estado de sesión: Sesión del usuario activa.
```

Versión corregida: ErrorAsignacionCorregido.java

```
package com.taller.practico;

public class ErrorAsignacionCorregido {
    public static void main(String[] args) {
        boolean cookieActiva = false;

        if (cookieActiva == true) {
            System.out.println("Estado de sesión: Sesión del usuario activa.");
        } else {
            System.out.println("Estado de sesión: Sesión inactiva.");
        }
    }
}
```

Salida corregida

```
Estado de sesión: Sesión inactiva.
```

Fragmento 2: Comparación Incorrecta de Cadenas de Texto (String)

```
package com.taller.practico;

public class ErrorComparacionString {
    public static void main(String[] args) {
        // Generación explícita de dos objetos String en posiciones distintas de
memoria
        String proveedor1 = new String("OneTrust");
        String proveedor2 = "OneTrust";

        // PUNTO DE INSPECCIÓN: Evalúe por qué '==' compara referencias de memoria
y no contenido
        if (proveedor1 == proveedor2) {
            System.out.println("Los proveedores coinciden exactamente.");
        } else {
            System.out.println("Los proveedores son distintos según la comparación
por referencia.");
        }
    }
}
```

✔ SOLUCIÓN: Fragmento 2

- **Error:** == compara referencias (si ambas variables apuntan al mismo objeto en memoria), no el contenido. `new String("OneTrust")` crea un objeto nuevo, distinto del literal `"OneTrust"` del pool de cadenas, aunque el texto sea idéntico.
- **Qué se ve en el depurador:** en Variables, `proveedor1` y `proveedor2` tienen el mismo texto pero identificadores de objeto distintos (`@...`). Con Evaluate Expression (Alt + F8), `proveedor1 == proveedor2` da `false` y `proveedor1.equals(proveedor2)` da `true`.
- **Corrección:** usar `equals` para comparar el contenido.

Salida con el error

```
Los proveedores son distintos según la comparación por referencia.
```

Versión corregida: ErrorComparacionStringCorregido.java

```
package com.taller.practico;

public class ErrorComparacionStringCorregido {
    public static void main(String[] args) {
        String proveedor1 = new String("OneTrust");
        String proveedor2 = "OneTrust";

        if (proveedor1.equals(proveedor2)) {
            System.out.println("Los proveedores coinciden exactamente.");
        } else {
            System.out.println("Los proveedores son distintos según la comparación
por referencia.");
        }
    }
}
```

Salida corregida

```
Los proveedores coinciden exactamente.
```

Fragmento 3: Caída en Cascada Involuntaria (Fall-Through) en switch Tradicional

```
package com.taller.practico;

public class ErrorFallThrough {
    public static void main(String[] args) {
        int opcionCategoria = 1;

        // PUNTO DE INSPECCIÓN: Observe la ejecución secuencial al omitir
sentencias de parada
        switch (opcionCategoria) {
```

```
            case 1:
                System.out.println("Categoría Selección: Lunes (Jornada Laboral)");
            case 2:
                System.out.println("Categoría Selección: Martes (Jornada
Laboral)");
            case 3:
                System.out.println("Categoría Selección: Miércoles (Jornada
Laboral)");
            default:
                System.out.println("Día no identificado");
        }
    }
}
```

✔ SOLUCIÓN: Fragmento 3

- **Error:** los `case` no tienen `break`. Al coincidir con `case 1`, Java sigue ejecutando todas las instrucciones siguientes (`case 2`, `case 3` y `default`) hasta terminar el `switch`.
- **Qué se ve en el depurador:** con F8, tras la primera impresión el cursor salta a la línea del `case 2`, luego al `case 3` y al `default`, en vez de salir del `switch`.
- **Corrección:** agregar `break;` al final de cada `case`. Alternativa: usar el `switch` moderno con `->`, que no tiene fall-through.

Salida con el error (imprime 4 líneas)

```
Categoría Selección: Lunes (Jornada Laboral)
Categoría Selección: Martes (Jornada Laboral)
Categoría Selección: Miércoles (Jornada Laboral)
Día no identificado
```

Versión corregida: ErrorFallThroughCorregido.java

```
package com.taller.practico;

public class ErrorFallThroughCorregido {
    public static void main(String[] args) {
        int opcionCategoria = 1;

        switch (opcionCategoria) {
            case 1:
                System.out.println("Categoría Selección: Lunes (Jornada Laboral)");
                break;
            case 2:
                System.out.println("Categoría Selección: Martes (Jornada
Laboral)");
                break;
            case 3:
```

```
                System.out.println("Categoría Selección: Miércoles (Jornada
Laboral)");
                break;
            default:
                System.out.println("Día no identificado");
        }
    }
}
```

Salida corregida

```
Categoría Selección: Lunes (Jornada Laboral)
```

Fragmento 4: Pérdida de Precisión por División Entera

```
package com.taller.practico;

public class ErrorPrecision {
    public static void main(String[] args) {
        int tareasCompletadas = 3;
        int tareasTotales = 4;

        // PUNTO DE INSPECCIÓN: Verifique por qué (3 / 4) trunca los decimales a 0
antes de multiplicar por 100
        // PISTA DE SOLUCIÓN: Utilice un moldeo de tipo '(double)' o cambie un
literal a flotante '4.0'
        double porcentajeAvance = (tareasCompletadas / tareasTotales) * 100;

        System.out.println("El porcentaje de avance es: " + porcentajeAvance +
"%");
    }
}
```

✓ SOLUCIÓN: Fragmento 4

- **Error:** `tareasCompletadas` y `tareasTotales` son `int`, así que `3 / 4` es una división entera que da 0. Ese 0 se multiplica por 100 y el resultado es 0.0, aunque la variable de destino sea `double`: la conversión se hace después de dividir.
- **Qué se ve en el depurador:** con Evaluate Expression (Alt + F8), `tareasCompletadas / tareasTotales` da `0`, mientras que `(double) tareasCompletadas / tareasTotales` da `0.75`.
- **Corrección:** convertir uno de los operandos a `double` antes de dividir, con `(double)` o usando un literal flotante como `4.0`.

Salida con el error

```
El porcentaje de avance es: 0.0%
```

Versión corregida: ErrorPrecisionCorregido.java

```
package com.taller.practico;

public class ErrorPrecisionCorregido {
    public static void main(String[] args) {
        int tareasCompletadas = 3;
        int tareasTotales = 4;

        double porcentajeAvance = ((double) tareasCompletadas / tareasTotales) *
100;

        System.out.println("El porcentaje de avance es: " + porcentajeAvance +
"%");
    }
}
```

Salida corregida

```
El porcentaje de avance es: 75.0%
```

Guía Paso a Paso para la Depuración (Debugging) en IntelliJ IDEA:

1. **Colocación de Breakpoints (Puntos de Interrupción):**
   - En el editor de código, haga clic sobre el margen izquierdo (a la derecha del número de línea) en la primera línea dentro del método `main`. Aparecerá un círculo rojo que marca la pausa de la ejecución.
2. **Ejecución en Modo Debug:**
   - Haga clic derecho sobre la clase con el error y seleccione la opción Debug 'NombreDeLaClase.main()' (o presione `Shift + F9`). La ejecución se detendrá en la línea marcada en resaltado amarillo.
3. **Avance Paso a Paso (Step Over):**
   - Presione la tecla **F8** (Step Over) para ejecutar la instrucción actual y avanzar a la siguiente línea.
   - Observe la pestaña Variables en el panel inferior del IDE para visualizar cómo cambian los valores en tiempo real dentro de la memoria.
4. **Inspección de Expresiones (Evaluate Expression):**
   - Presione `Alt + F8` sobre cualquier variable o expresión condicional para abrir la calculadora de expresiones y verificar qué valor devuelve exactamente la JVM.

Lista de Verificación Final (Checklist de Autoevaluación)

Valide de manera autónoma el cumplimiento de cada uno de los objetivos de este taller práctico:

	Objetivo	Evidencia
☑	El proyecto `TallerUnidad1_2` está creado en IntelliJ IDEA dentro del paquete `com.taller.practico` y contiene la clase `Main`.	Proyecto Taller_Java con el paquete `com.taller.practico` y la clase `Main` (Sección 1 de esta guía).
☑	Se completó la predicción manual de la tabla de la Sección 1 y se verificaron los resultados exactos ejecutando el código en el IDE.	Tabla completa con predicción y resultado real; coinciden en las 4 filas.
☑	Se implementaron exitosamente los 3 retos de decisión condicional (`if-else`) de la Sección 2 (Clasificación de Edades, Aprobación de Notas y Tarifas).	`ClasificacionEdades`, `SistemaAprobacion` y `TarifasEntrada`, cada una con su `main`.
☑	Se refactorizó la asignación de variable a una expresión compacta utilizando el operador ternario (`? :`).	`OperadorTernario`: una sola línea con `cookieSesionActiva ? "ACTIVA" : "INACTIVA"`.
☑	Se construyó el menú de la Sección 3 en sus dos variantes: `switch` tradicional con `break` y `switch` moderno con notación de flecha (`->`).	`DiaSemanaTradicional` y `DiaSemanaModerno`.
☑	El Reto Integrador de la Sección 4 produce con precisión matemática los totales de 638.00 para el Caso A y **275.50** para el Caso B.	`SimuladorCajaInventario` imprime Total a pagar = $638.00 y $275.50.
☑	Se identificaron y solucionaron las causas técnicas de los 4 fragmentos defectuosos en la Sección 5 empleando las herramientas del Debugger (F8 y Breakpoints).	Diagnóstico y versión corregida de cada fragmento (asignación vs comparación, `equals`, `break`, división entera).